

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Introducción a la programación sobre GPUs

Universidad Nacional de La Plata



Facultad de Informática





1

Concepto GPGPU y era Manycore

2

GPUs como arquitectura paralela

3

GPUs de Nvidia

Sistema de procesamiento. Jerarquía de memoria.

4

Modelo de programación Nvidia CUDA

5

Métricas en GPU

6

Optimizaciones Nvidia CUDA

7

Ocultamiento de latencia – CUDA Streams

8

Múltiples GPUs y modelos híbridos

9

Interoperabilidad

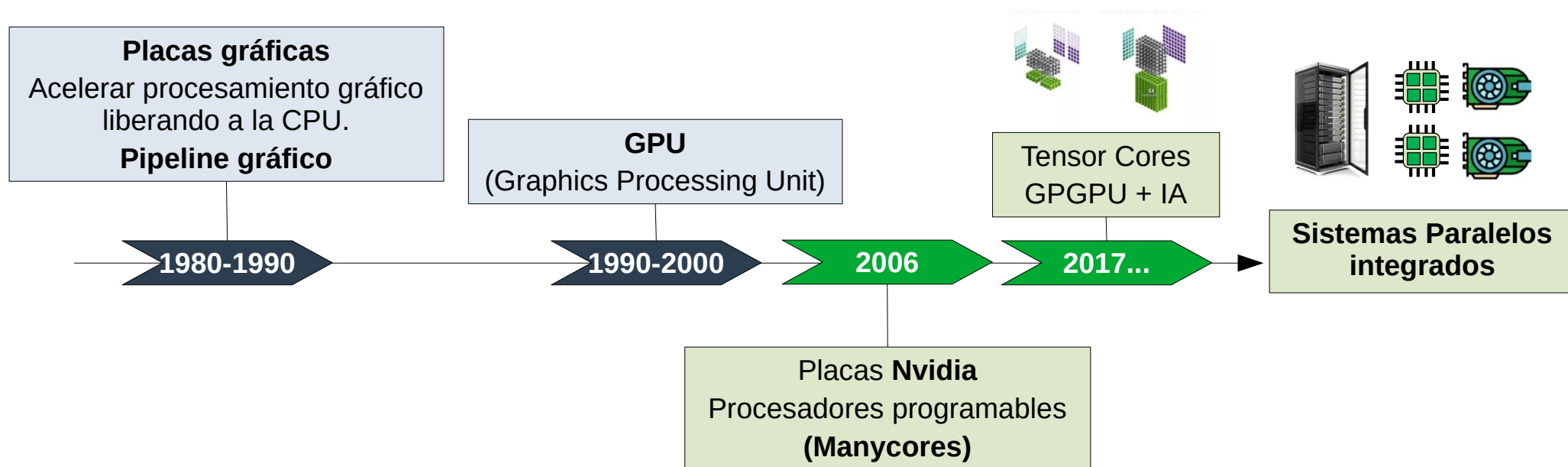


1

Concepto GPGPU y era Manycore

Concepto GPGPU y era Manycore

4



General-Purpose Computing on Graphics Processing Units (GPGPU): Concepto según el cual se desarrollan aplicaciones de propósito general sobre unidades de procesamiento gráfico (GPUs).



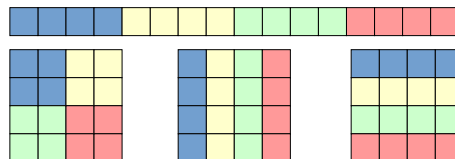
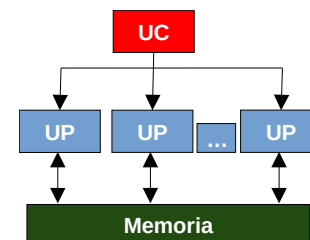
2

GPUs como arquitectura paralela

GPUs como arquitectura paralela

6

- Arquitectura de **memoria compartida**:
 - Posee un conjunto de unidades de procesamiento que comparten distintos niveles de la jerarquía de memoria propia de la GPU
 - **NO se comparte la memoria RAM de la máquina donde se aloja.**
- **Modelo SIMD** (o STMD)
- Mas adecuada para:
 - **Paralelismo de datos** sobre **datos regulares** en aplicaciones **intensivas en CPU**





3

GPUs de Nvidia

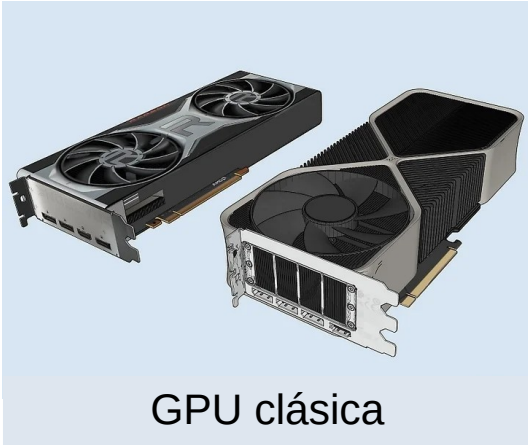
Sistema de procesamiento

Jerarquía de memoria

GPUs Nvidia

8

- Generalmente, una GPU Nvidia se conecta vía puerto PCI express:



GPUs Nvidia

9

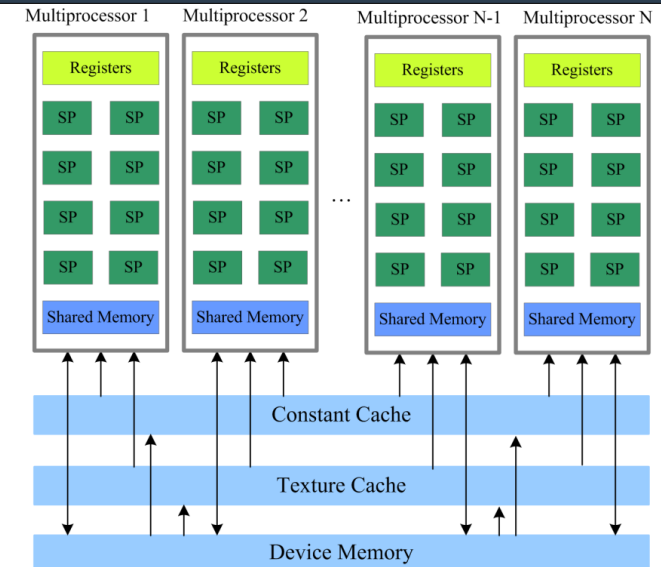
- Internamente, una **GPU Nvidia** está compuesta por:
 - Unidades de procesamiento simples agrupadas que forman el **sistema de procesamiento**
 - Una jerarquía de memoria compleja que forma el **sistema de memoria**



Grupo de cores



Jerarquia de Memoria





3

GPUs de Nvidia

Sistema de procesamiento

Jerarquía de memoria

GPUs de Nvidia

Sistema de procesamiento

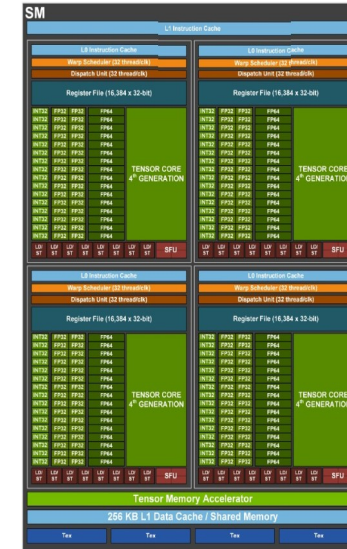
11

- La estructura de las GPUs Nvidia puede variar de un modelo a otro pero diferentes arquitecturas tienen componentes comunes.
- Las unidades de procesamiento se agrupan en multiprocesadores llamados *Stream multiprocessors* o **SM**.

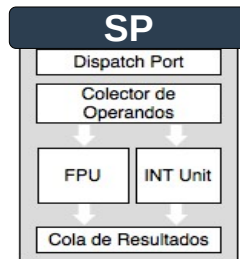
Modelos 2010 - 2016



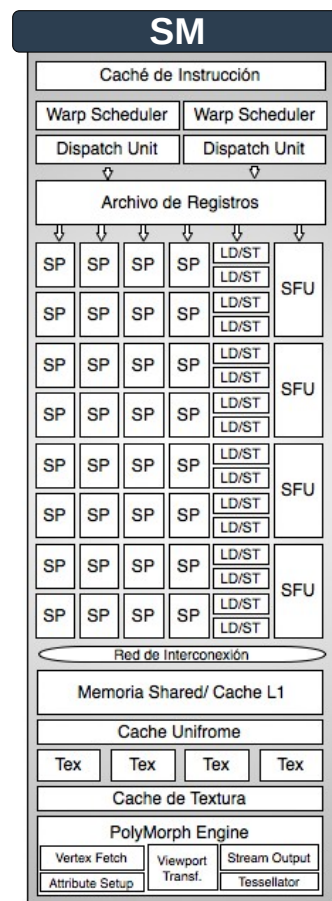
Modelos 2017 - Actualidad



- Cada **SM**:
 - 32 Unidades simples (**cores**): *Scalar processors* o **SPs**
 - 16 Unidades de **Load/Store**:
 - Escriben o leen direcciones de 16 hilos por ciclo
 - 4 Unidades de Funciones Especiales o **SFUs**
 - Planificadores de Hilos o *Warp Schedulers*
- SP unidad muy simple compuesta por:
 - Una unidad de punto flotante (**FPU**) simple y doble precisión
 - Una unidad aritmético lógica (**ALU**)



Modelos 2010 - 2016

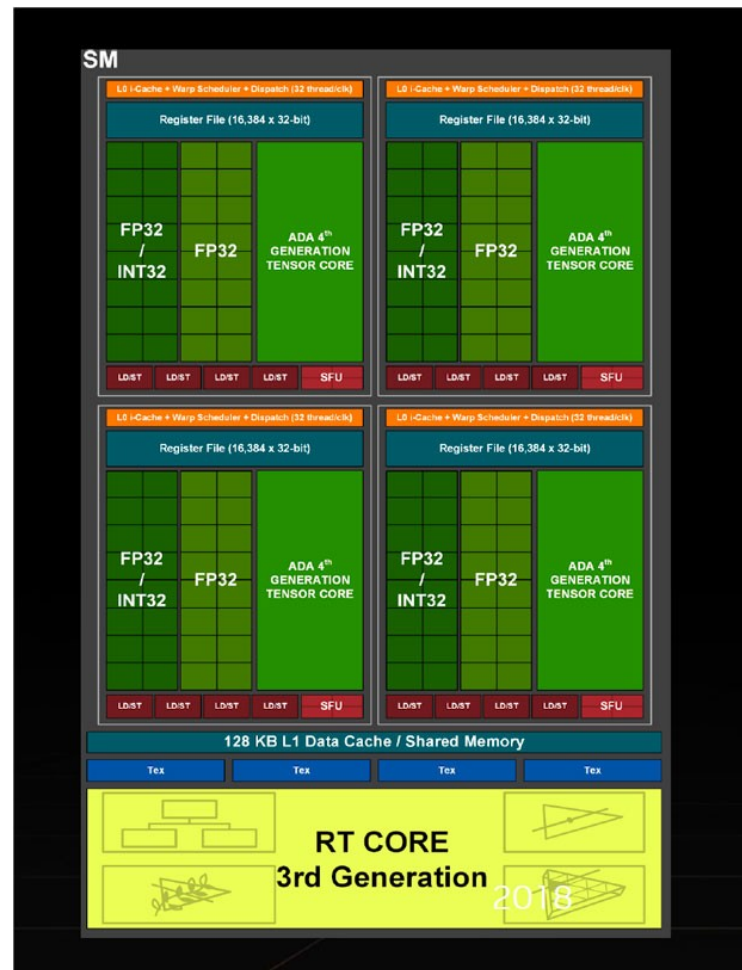


- Cada **SM** 4 bloques (Execution Partition).
- Cada bloque:
 - 32 cores FP32
 - 16 cores FP64
 - 16 cores INT32
 - 1 Tensor core
 - 4 SFUs
 - 8 Unidades de Load/Store

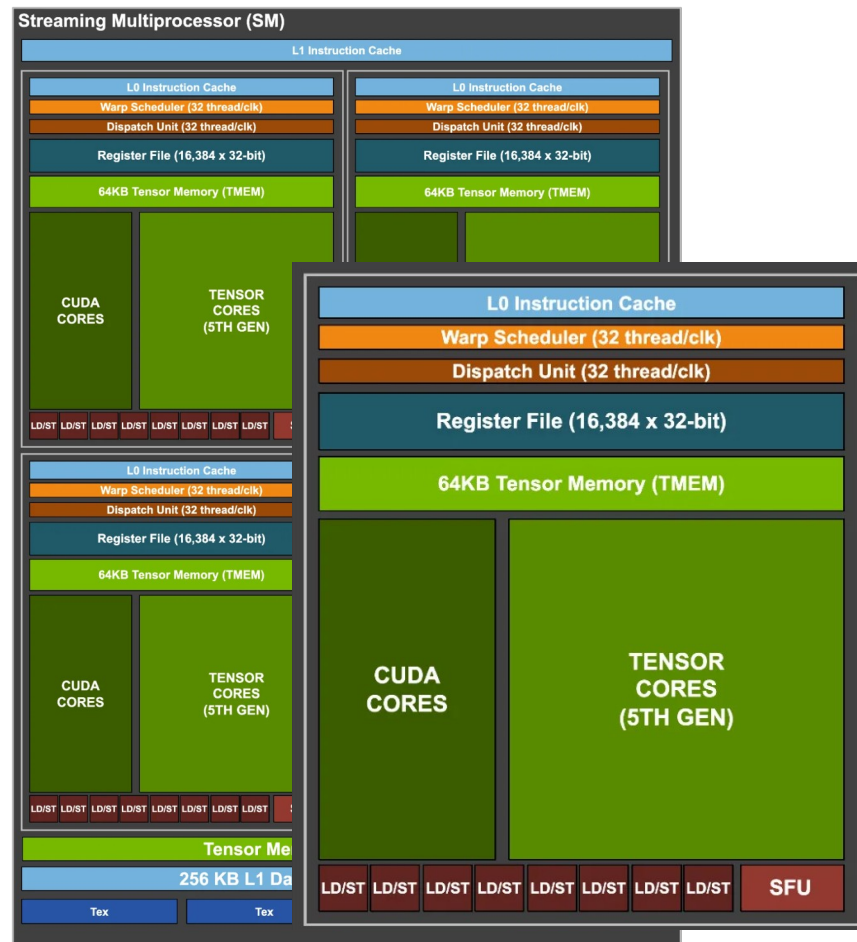
Modelos 2017 - Actualidad



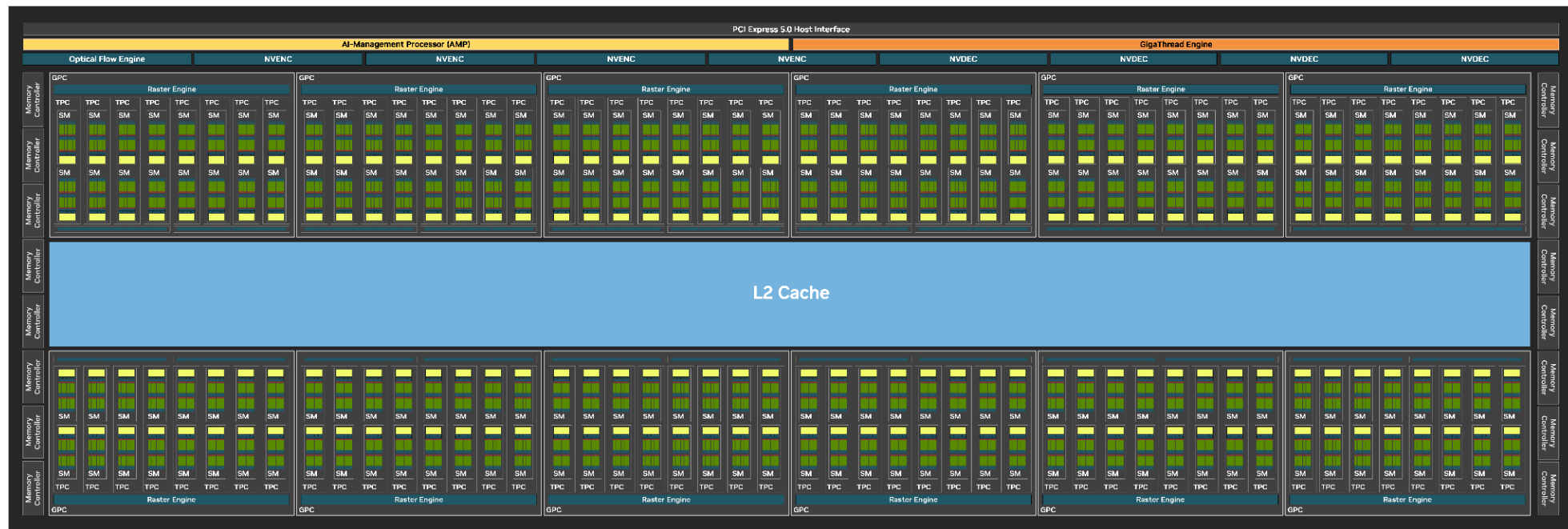
- Cada **SM** 4 bloques (Execution Partition).
- Cada bloque:
 - 32 Cuda cores:
 - 16 cores específicos FP32
 - 16 cores unificados (FP32 o INT32)
 - **FP64 resuelve en FP32**
 - 1 Tensor core
 - 4 SFUs
 - 4 Unidades de Load/Store



- Cada **SM** 4 bloques (Execution Partition).
- Cada bloque:
 - 32 Cuda cores unificados (FP32 o INT32)
 - **FP64 resuelve en FP32**
 - 1 Tensor core
 - 4 SFUs
 - 4 Unidades de Load/Store



- Nvidia microarquitectura Blackwell (RTX 5090):
 - 24576 Cores
 - 768 Tensor Cores





3

GPUs de Nvidia

Sistema de procesamiento

Jerarquía de memoria

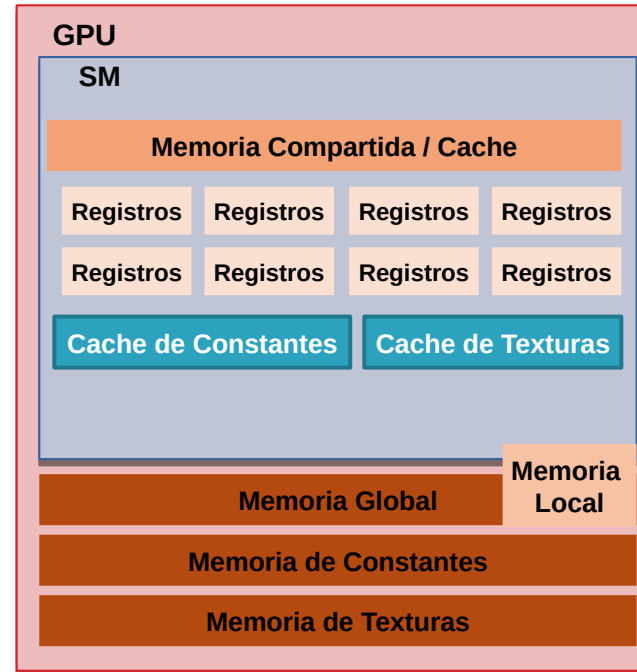
Arquitectura Nvidia

Sistema de memoria

18

- El sistema de memoria se organiza en una jerarquía compleja que puede clasificarse:

- **On-Chip:** Dentro de cada SM
 - Registros
 - Memoria compartida (shared)
 - Distintas caché
- **Off-Chip:** Fuera de los SMs
 - Memoria global
 - Memoria de constantes
 - Memoria de texturas
 - Memoria local



- El costo de acceso a las memorias **On-Chip (más rápidas)** es menor al costo de acceso a las memorias **Off-Chip (más lentas)**.

A dark, industrial-style workspace. In the foreground, a desk holds an open book with dense text, a pen, and some papers. In the background, a glowing monitor displays data, and the room is filled with various electronic equipment and warm, ambient lighting.

4

Modelo de programación Nvidia CUDA

Modelo de programación Nvidia CUDA

20

- A partir de 2006 Nvidia provee **CUDA** para facilitar la programación de sus GPUs.

Compute Unified Device Architecture (CUDA): plataforma para cómputo paralelo que incluye un **compilador** y un conjunto de herramientas de desarrollo que permiten a los programadores usar una **extensión del lenguaje de programación C** para implementar algoritmos sobre GPUs de NVidia.

- A tener en cuenta:
 - **Copia explícita de memoria:** Los datos deben copiarse explícitamente (lo hace el programador) desde la memoria RAM de la máquina (CPU) a la memoria de la GPU, y viceversa.
 - **Organización de hilos:** en grupos (grids, bloques, warps)
 - Todos los **hilos** ejecutan el **mismo código (STMD)** - Modelo Flynn **SIMD**
 - La **unidad de planificación** no es un hilo individual sino un **grupo de hilos** que se asignan a los Sms.
 - Las GPU **no** puede hacer **entrada/salida**





5

Métricas en GPU

- No es posible ejecutar un programa secuencial en una GPU:
 - Speedup respecto al mejor algoritmo secuencial ejecutado sobre CPU (**Speedup relativo**).
 - Speedup respecto a otras arquitecturas paralelas (multicores, clusters, etc.)
- Usando una única GPU:
 - La idea de **Speedup ideal** no queda bien definida por características inherentes al cómputo.
 - Se crean mas hilos que unidades de procesamiento (multithreading por hardware).
 - Difícil (o imposible) evaluar Amdahl, Gustafson y Barsis, y el concepto de escalabilidad.
- Habitualmente, se considera una única GPU como una única unidad de procesamiento.
 - ¿Cuánto se escala al agregar mas GPUs?
- Conveniencia de utilizar otro tipo de métricas como el **Throughput** (GFLOPs)





Optimizaciones Nvidia CUDA

24



- El modelo básico de programación en CUDA permite obtener beneficios de las GPUs.
- Sin embargo, conociendo las características y los factores limitantes del rendimiento de las GPU podemos alcanzar mayor rendimiento.
- Podemos aplicar optimizaciones para explotar al máximo el paralelismo de la arquitectura subyacente:

- Relacionadas a la memoria:

- Organización de los accesos (**Coalescence**)
- Técnicas de carga anticipada de datos (**Prefetching**).



- Relacionadas a la ejecución de los hilos (**Divergence**).
- Relacionadas al rendimiento de las instrucciones: mezcla y granularidad.
- Relacionadas a la asignación de recursos en un SM (**Occupancy**).



Ocultamiento de latencia – CUDA Streams

26

- Flujo de procesamiento típico sobre GPU:

- 1) Copiar los datos de entrada de CPU a GPU
- 2) Realizar la ejecución sobre GPU
- 3) Copiar los resultado de GPU a CPU



- Las transferencias de datos desde y hacia la GPU son costosas.
- **CUDA Streams:** técnica para **ocultar la latencia** de estas transferencias, solapando el cómputo con la copia de datos en ambos sentidos.
- CUDA Stream también permite ejecutar varias aplicaciones CUDA simultáneamente sobre una única GPU cuando cada una de ellas no utiliza la GPU al 100%.

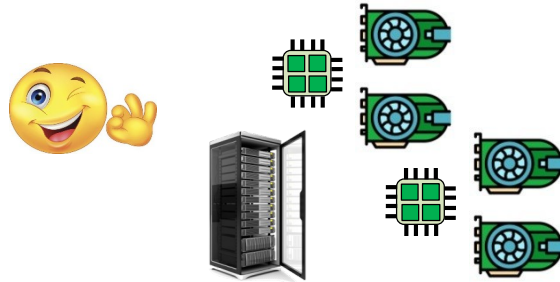




Múltiples GPUs y modelos híbridos

28

- Actualmente, un máquina puede incorporar múltiples GPUs y utilizarlas para resolver problemas específicos.
- Estas máquinas suelen ser multicores muy potentes y podrían estar conectadas con otras máquinas similares conformando un cluster.
- Como resultado, tenemos una **arquitectura paralela híbrida heterogénea** muy potente y podemos utilizar un modelo de programación híbrido que incluya las GPUs:
 - Pthreads + CUDA
 - OpenMP + CUDA
 - MPI + Pthreads + CUDA
 - MPI + OpenMP + CUDA



Complejidad adicional:

- Compilación
- Ejecución
- Distribución de carga: GPUs y CPUs poseen diferente potencia de cómputo





- Las herramientas de desarrollo de aplicaciones paralelas, incluido CUDA, suelen estar escritas en lenguajes como **C/C++** o **Fortran**.
-  • De todos modos, aplicaciones escritas en otros lenguajes utilizan CUDA para alcanzar mayor rendimiento. Esto se logra a partir del concepto de **interoperabilidad**.

Interoperabilidad entre lenguajes de programación: Capacidad de distintos lenguajes de programación para interactuar, integrarse y colaborar dentro de una misma aplicación o sistema, permitiendo que componentes escritos en lenguajes diferentes puedan comunicarse, compartir datos, y aprovechar sus fortalezas respectivas.

- Algunas posibilidades:
 - Ejecución independiente y comunicación mediante archivos
 - Bibliotecas estáticas o dinámicas
 - Código embebido, wrappers o APIs



